

Table of Contents

PURPLE-LOOP

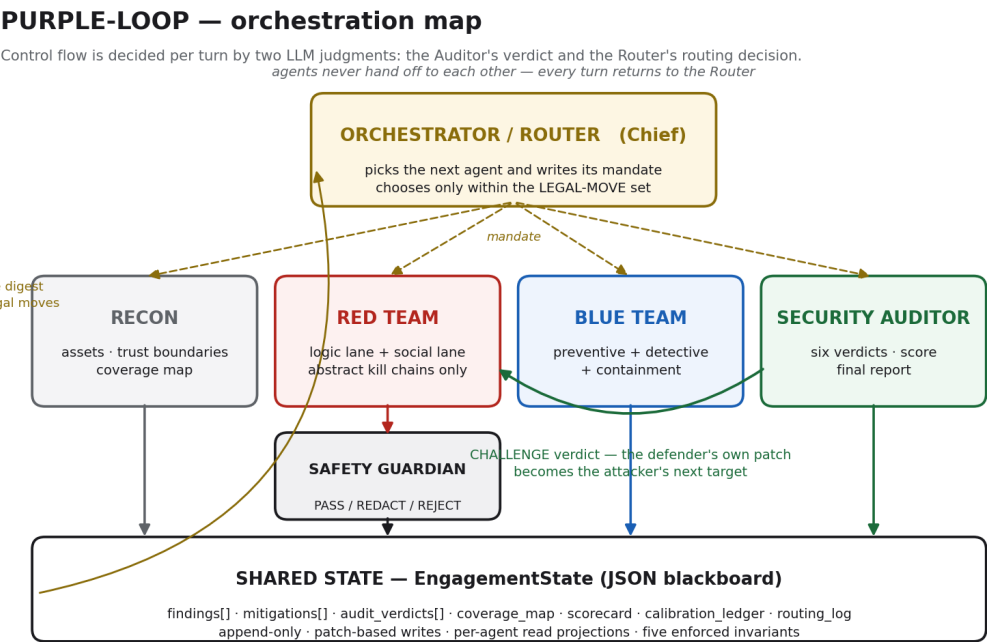
A Dynamic Multi-Agent Orchestration Architecture for Adversarial Security Evaluation

Course: Orchestration — Capstone Project **Topic:** Cybersecurity Attack / Defense Simulation (Red vs Blue) **Platform target:** Myskillos (myskillos.com) — agent definitions are platform-portable

0. Compliance Map — brief requirement -> where it is satisfied

Requirement (as stated in the brief)	Where it is satisfied	Verifiable how
Dynamic multi-agent orchestration architecture	Section 2 roster, Section 5 routing, orchestration map below	--self-test check 10 asserts the executed agent sequence is not a fixed cycle
LLM agents critique one another	Section 4 Loop B: six-verdict cross-critique; Blue's patch becomes Red's target	Report Section 4 "Disagreement history" — every verdict with its rationale
LLM agents audit one another	Independent security-auditor with separation of powers (Section 3.2 invariant 3)	--self-test check 3: the Auditor is structurally prevented from authoring findings
LLM agents make decisions with one another	Section 5: the Router's next move is an LLM decision inside a constrained legal-move set	Report Section 2 "Routing history" — legal moves vs. chosen move, per round
Not static/classic if-else code	Deterministic code only <i>validates</i> moves; it never chooses them	Section 7 rounds 5, 8 and 9–11 are unreachable in a pipeline
Built with Claude, uploaded to Myskillos	Section 11: importable CLAUDE.md + .claude/agents/*.md package, generated from this document	python purple_loop.py --build-package
Red Team: social engineering and software logic	red-team runs two lanes; the Router switches lanes on novelty collapse	Report Section 3 — lane column per finding

Red Team devises an attack scenario	attack_narrative per finding: abstract kill chain + MITRE ATT&CK IDs	Report Section 3 and final_state.json
Blue Team develops patches and countermeasures	Three mandatory layers: preventive / detective / containment	Report Section 3; missing layers are scored down, not silently accepted
Audit agent produces final security score	Section 6 weighted rubric, 0–100 with grade bands	Report Section 1 "Score breakdown"
Audit agent produces the report	The closing turn is the auditor authoring the report over the finished state	Report Section 0 — written by the agent, not by the harness
State Management	Section 3	--self-test checks 1, 2, 8
Reflection & Feedback Loops	Section 4	--self-test check 9 (challenge -> durability bonus)
Dynamic Routing	Section 5	--self-test checks 9, 10



PURPLE-LOOP orchestration map

1. Problem Statement

A classic security-review script is static: it walks a checklist, matches patterns with if/else, and emits a score. It cannot reason about *novel* attack composition, it cannot argue with itself, and it cannot decide that a question deserves a second look.

PURPLE-LOOP replaces that with a debate. Three primary LLM agents — an attacker, a defender, and an auditor — are placed in an adversarial loop over a shared state object. None of them decides how long the engagement runs or who speaks next; a **Router agent** makes that call at every turn based on the *current epistemic state* of the engagement. The system halts only when the auditor certifies convergence, or when the round budget is exhausted.

The output is not a checklist. It is a negotiated security posture assessment with a documented disagreement history.

Scope and safety. This is a *tabletop* simulation. The Red Team agent reasons at the level of vulnerability **classes**, MITRE ATT&CK **technique IDs**, and kill-chain **narratives** against a fictional architecture supplied as text. A dedicated Safety Guardian agent enforces that no agent ever emits working exploit code, live payloads, malware, or operational intrusion instructions. Attack reasoning that cannot be expressed abstractly is redacted, not produced. This constraint is an architectural component, not a disclaimer.

2. Agent Roster

#	Agent	Role	Reads from state	Writes to state
0	ORCHESTRATOR (Router)	Meta-agent. Chooses the next agent and the <i>mandate</i> it receives. Never produces security content.	full state (summarized)	routing_log, next_agent, mandate
1	RECON	Decomposes the target architecture into an asset/trust-boundary/data-flow inventory. Runs once, plus on demand when Red claims missing information.	target_spec	asset_inventory, trust_boundaries, coverage_map
2	RED	Attacker. Produces attack hypotheses across two lanes: <i>software/logic</i> and <i>social engineering / human process</i> .	asset_inventory, mitigations, audit_verdicts	findings[]
3	BLUE	Defender. Produces layered countermeasures: preventive patch,	findings[], audit_verdicts	mitigations[]

		detection rule, and containment/blast-radius control per finding.		
4	AUDITOR	Independent judge. Scores each finding/mitigation pair, detects unaddressed residual risk, and issues the verdict that drives the loop. Produces the final report.	everything	audit_verdicts[], scorecard, convergence
5	SAFETY GUARDIAN	Cross-cutting filter. Inspects every RED output before it enters shared state. Redacts operational detail, rewrites to abstract form, or rejects.	candidate RED output	guardian_log, sanitized finding

Agents 2 and 3 never talk to each other directly. **All inter-agent communication is mediated by shared state**, which makes the system inspectable and replayable.

3. State Management

3.1 The blackboard

A single JSON object, EngagementState, is the only channel between agents. Each agent receives a *projection* of it (only the fields it is entitled to read), and returns a **patch**, never a full rewrite. Patches are applied by the orchestrator with an append-only history, so any turn can be replayed or diffed.

```
{
  "engagement_id": "eng-2026-001",
  "round": 3,
  "max_rounds": 6,
  "phase": "CHALLENGE",

  "target_spec": "<free-text system architecture supplied by the user>",

  "asset_inventory": [
    { "id": "A1", "name": "Customer PII store", "tier": "crown_jewel",
      "exposure": "internal", "owner": "platform" }
  ]
}
```

```

],
"trust_boundaries": [
  { "id": "TB1", "from": "public internet", "to": "API gateway",
    "controls": ["WAF", "rate limit"] }
],

"coverage_map": {
  "STRIDE": { "spoofing": 0.8, "tampering": 0.4, "repudiation": 0.0,
    "info_disclosure": 0.9, "dos": 0.3, "eop": 0.6 },
  "human_layer": { "phishing": 0.7, "pretexting": 0.2,
    "insider": 0.0, "vendor_chain": 0.1 }
},

"findings": [
  { "id": "F3", "round_introduced": 2, "lane": "logic",
    "title": "Password-reset token not bound to session",
    "attack_narrative": "<abstract kill-chain, no payloads>",
    "attck": ["T1078", "T1556"],
    "assets_hit": ["A1"],
    "likelihood": 4, "impact": 5, "confidence": 0.7,
    "status": "CONTESTED",          // OPEN | MITIGATED | CONTESTED |
RESIDUAL | DISMISSED
    "novelty": 0.8,
    "supersedes": ["F1"]
  }
],

"mitigations": [
  { "id": "M3", "finding_id": "F3", "round": 2,
    "preventive": "...", "detective": "...", "containment": "...",
    "cost": "medium", "residual_risk_claim": 0.2,
    "assumptions": ["IdP supports token binding"] }
],

"audit_verdicts": [
  { "round": 2, "finding_id": "F3", "verdict": "INSUFFICIENT",
    "rationale": "Detection rule covers only the happy path; no control for the
vendor-support channel.",
    "residual_risk_assessed": 0.55,
    "instruction_to": "RED",
    "mandate": "Attack the proposed patch M3 directly; assume it is deployed." }
],

"scorecard": {
  "attack_surface_coverage": 0.72,
  "defense_depth": 0.61,
  "detection_coverage": 0.45,
  "blast_radius_control": 0.58,
  "residual_exploitability": 0.34,
  "composite": 63.4, "grade": "C+"
}

```

```

},

"convergence": {
  "new_findings_last_round": 1,
  "unresolved_count": 2,
  "score_delta": 4.1,
  "converged": false
},

"routing_log": [
  { "round": 2, "chose": "BLUE", "why": "2 OPEN findings with no mitigation; Red novelty falling." }
],
"guardian_log": [
  { "round": 2, "target": "F3", "action": "REDACT",
    "note": "Removed step-level request construction; kept abstract technique reference." }
]
}

```

3.2 State invariants (enforced by the orchestrator)

1. **Append-only findings.** A finding is never deleted; it is DISMISSED or superseded, with a reason. The disagreement history is part of the deliverable.
2. **Every mitigation must reference a live finding.** Orphan defenses are rejected — this stops Blue from "defending" imaginary problems to inflate the score.
3. **The Auditor cannot author findings or mitigations.** Separation of powers: the judge may only score, contest, and instruct.
4. **Only the Guardian may write sanitized RED content.** Red's raw output never lands in shared state directly.
5. **Monotone round counter.** No agent can extend max_rounds; only the human operator can.

3.3 Context economy

Full state grows fast. Each agent receives: - its entitled fields in full for the **current** round, - a rolling **compaction** of prior rounds (Auditor-generated one-line summaries per finding), - never the raw transcript of other agents' reasoning.

This keeps token cost roughly flat as rounds increase, and forces agents to communicate through structured claims rather than prose.

4. Reflection & Feedback Loops

Three nested loops operate at different timescales.

Loop A — Intra-agent reflection (within one turn)

Every content agent runs a **draft** -> **self-critique** -> **revise** cycle inside a single turn before emitting its patch.

- **RED** drafts findings, then self-critiques against: *Is this actually reachable given the trust boundaries? Am I assuming a control that doesn't exist? Is this a restatement of an existing finding?* Findings that fail self-critique are dropped with a logged reason.
- **BLUE** drafts a mitigation, then attacks its own patch: *What does this control NOT stop? What new failure mode does it introduce? What does it assume about the environment?* The surviving assumptions become the assumptions[] field — which the Auditor later uses as attack surface.
- **AUDITOR** drafts a score, then checks it for anchoring: *Would I give the same score if this finding had arrived in round 1 instead of round 4?*

Loop B — Adversarial cross-critique (round-level)

This is the core engine. The Auditor's verdict on each finding/mitigation pair selects the loop's next move:

Verdict	Meaning	Loop consequence
SOUND	Mitigation credibly closes the finding	Finding -> MITIGATED; freed budget goes to coverage gaps
INSUFFICIENT	Patch is partial or shallow	Back to BLUE with a specific deficiency to fix
CHALLENGE	Patch is plausible but untested	To RED with mandate "assume this patch is live — bypass it"
OVERCLAIM	Blue's residual-risk claim is not supported	Blue must re-state with evidence; score penalty recorded
SPECULATIVE	Red's finding is not reachable / unsupported	Finding -> DISMISSED, Red's confidence calibration penalized
RESIDUAL_ACCEPT	Real risk, no viable mitigation at reasonable cost	Recorded as accepted residual risk in the report

The CHALLENGE verdict is what makes this a genuine loop rather than a pipeline: **Blue's own patch becomes Red's next target**. A defense that survives two challenge rounds earns a durability bonus in the final score.

Loop C — Convergence control (engagement-level)

After each round the orchestrator computes:

```
converged = new_findings_last_round == 0
            AND unresolved_count == 0
            AND |score_delta| < 3.0
```

...or round $\geq$ max_rounds. If Red's novelty average drops below 0.25 for two consecutive rounds, the orchestrator forces a **lane switch** (logic $\leftrightarrow$ social engineering) before allowing convergence — preventing premature agreement, which is the classic failure mode of self-critiquing agent systems.

Calibration feedback. Confidence scores are scored against outcomes: a Red finding claimed at 0.9 confidence that is later DISMISSED lowers Red's calibration multiplier, which the Auditor applies as a discount on subsequent Red claims. Agents are thus held accountable across rounds, not just within a turn.

5. Dynamic Routing

5.1 Why not if/else

A static flow (Recon $\rightarrow$ Red $\rightarrow$ Blue $\rightarrow$ Audit $\rightarrow$ end) wastes turns on solved areas and starves contested ones. PURPLE-LOOP instead treats routing as a **decision made by an LLM under constraints**.

5.2 The mechanism

At each turn the Router receives a compact **state digest** (not the full state) and must return:

```
{ "next_agent": "RED",  
  "mandate": "Focus exclusively on the human layer; insider and vendor-chain are  
at 0.0 coverage.",  
  "scope_filter": ["A1", "A4"],  
  "rationale": "Logic lane saturated (novelty 0.18); coverage_map shows two  
untouched human vectors.",  
  "expected_state_change": ">=1 new finding in human_layer,  
coverage_map.human_layer.insider > 0"  
}
```

Three properties make this safe and non-arbitrary:

1. **Constrained action space.** The orchestrator computes the set of *legal* moves from state invariants (e.g. BLUE is illegal when there are no OPEN findings; AUDITOR is illegal before at least one mitigation exists this round). The Router chooses only within that set — LLM judgment inside a formally bounded space.
2. **Mandate, not just destination.** The Router doesn't only pick *who* speaks, it writes *what they must accomplish*. Two calls to RED in the same engagement are different agents in effect, because the mandate reshapes their objective.
3. **Justification + falsifiable prediction.** Every routing decision carries a rationale and an expected_state_change. Next round the orchestrator checks whether the prediction held. Repeated mispredictions push the Router toward different strategies — routing itself is under a feedback loop.

5.3 Routing heuristics available to the Router

The Router is *told* these signals and reasons over them; it is not hard-coded to them.

Signal

coverage_map cell near 0
OPEN findings, no mitigation
INSUFFICIENT verdicts pending
CHALLENGE verdicts pending
Red novelty < 0.25
Round budget nearly spent
Guardian rejection rate high

5.4 Escalation to the human

If the Router twice fails to find a legal, useful move — for example when the target_spec is too thin to support further analysis — it emits next_agent: "HUMAN" with a specific question. The engagement pauses rather than hallucinating depth it doesn't have.

6. Scoring Model (Auditor's rubric)

Composite score on 0–100, weighted:

Dimension	Weight	Definition
Attack surface coverage	20%	Fraction of STRIDE + human-layer cells meaningfully probed
Defense depth	25%	Mean layers per mitigated finding (preventive / detective / containment)
Detection coverage	20%	Fraction of findings with a concrete detection signal
Blast-radius control	15%	Containment quality for crown-jewel assets
Residual exploitability	20% (inverse)	Auditor-assessed post-mitigation risk, weighted by asset tier

Durability bonus: +3 per mitigation that survived a CHALLENGE round unbroken.

Overclaim penalty: -2 per OVERCLAIM verdict. **Calibration penalty:** -0.5 x (dismissed high-confidence findings).

Grade bands: A >= 85, B >= 75, C >= 65, D >= 55, F < 55.

7. Worked Trace — actual run, not illustrative

The trace below is the real output of `purple_loop.py` against `sample_target.md` (the fictional "NovaPay" fintech) with a 12-round budget. The full log is Appendix B; nothing in this table is hand-written.

R	Router chose	Why (routing rationale)	What happened
1	RECON	State has no asset inventory	7 assets, 4 trust boundaries. Three assets — admin console, log/backup pipeline, CI/CD secrets — were inferred , not named in the spec
2	RED (<i>lane: social</i>)	Human-layer coverage cells all at 0.0	F1 support-desk verification, F2 vendor standing access. A third draft was withheld by self-critique : reachability not established
3	BLUE	Two unmitigated findings; defender is behind	M1 (F1) — no detection layer, residual claimed 0.05; M2 (F2) — all three layers
4	AUDITOR	Mitigations await judgment	F1 -> INSUFFICIENT (blind if prevention fails); F2 -> CHALLENGE
5	RED (<i>mandate: bypass M2</i>)	An open CHALLENGE must be answered before scoring	F3 — bypass through M2's stated assumption "no legacy or emergency path bypasses the new control"
6	BLUE	F1 has an unresolved INSUFFICIENT verdict	M3 (F3); M4 (F1 v2 , detection added)
7	AUDITOR	Mitigations await judgment	F2 -> INSUFFICIENT (its challenge produced a working bypass); F3 -> CHALLENGE; F1 -> SOUND
8	RED (<i>mandate: bypass M3</i>)	Open CHALLENGE on F3	No finding produced. RED reports honestly that the v2 control closes the assumption gap
9	BLUE	F2 rework outstanding	M5 (F2 v2) — regresses, drops the detection layer again
10	AUDITOR	Mitigations await judgment	F3 -> SOUND, durability bonus +3 (survived a challenge unbroken); F2 -> INSUFFICIENT again

11	BLUE	F2 still unresolved	M6 (F2 v3) — all three layers
12	AUDITOR	Round budget nearly spent	F2 -> SOUND, +3. Final: 82.6 / 100 (B)

Three moments in this trace are impossible in a static pipeline:

- **Round 5** exists only because a verdict sent control *backwards* from judge to attacker.
- **Round 8** is a turn where the attacker produces nothing and that is the correct, scored outcome — the defense earned its durability bonus by surviving, not by being asserted sound.
- **Rounds 9–11** are a defender who regressed and was caught twice by the same rubric. A checklist scorer would have accepted M5.

The engagement did **not** converge: it ran out of budget with `attack_surface_coverage` at 0.30, because the challenge loops consumed the turns that would otherwise have probed the software-logic lane. That is an honest result, not a failure of the run — see Section 10.

8. Mapping to the Technical Expectations

Requirement	Where it lives
State Management	Section 3 — single JSON blackboard, patch-based writes, append-only history, per-agent read projections, five enforced invariants, context compaction
Reflection & Feedback Loops	Section 4 — Loop A intra-agent draft/critique/revise; Loop B six-verdict adversarial cross-critique with CHALLENGE re-routing; Loop C convergence + cross-round confidence calibration
Dynamic Routing	Section 5 — LLM router over a formally constrained legal-move set, mandate generation, falsifiable routing predictions, novelty-driven lane switching, human escalation
Not static if/else	Control flow is decided per-turn by two LLM judgments (Auditor verdict, Router decision). Deterministic code only <i>validates</i> moves; it never chooses them.

9. Files in this Deliverable

File	Purpose
01_ARCHITECTURE.md (this file)	Design document / report

02_AGENT_PROMPTS.md	Copy-paste system prompts for all six agents (Myskillos-ready)
03_STATE_SCHEMA.json	Formal JSON-Schema for EngagementState
04_SCORING_RUBRIC.md	The Auditor's full scoring rubric and anti-anchoring checks
purple_loop.py	Runnable orchestrator — mock mode, live API mode, self-test, package build
myskillos_package/	The importable agent team: CLAUDE.md + .claude/agents/*.md (Section 10)
architecture.png	The orchestration map reproduced in Section 0
sample_target.md	Example target architecture ("NovaPay", fictional)
sample_run_report.md	Output of the run traced in Section 7
final_state.json	The complete end-of-engagement state object from that run

The orchestrator loads its prompts directly from 02_AGENT_PROMPTS.md, so the document and the implementation cannot drift apart — if a prompt is renamed or removed, the program refuses to start.

How to run

```
python purple_loop.py --self-test          # assert the architectural invariants
python purple_loop.py --target sample_target.md  # full engagement, no API
key needed
python purple_loop.py --target sample_target.md --live # same, driven by real LLM
agents
python purple_loop.py --build-package      # emit the Myskillos-importable
team
```

--self-test turns every claim in Section 3.2 into an executable assertion:

```
PASS invariant 1 — findings are append-only, supersede marks not deletes
PASS invariant 2 — orphan mitigations rejected
PASS invariant 3 — Auditor cannot author findings
PASS invariant 4 — RED output cannot bypass the Safety Guardian
PASS verdict vocabulary is enforced
PASS Guardian redacts operational detail
PASS all six agent prompts load from 02_AGENT_PROMPTS.md
PASS agents receive projections, not the whole blackboard
PASS full engagement: budget, legal moves, challenge->durability
PASS control flow is not a fixed pipeline
10/10 checks passed
```

The last check is the important one for grading: it asserts that the executed agent sequence is **not** the fixed cycle RECON -> RED -> BLUE -> AUDITOR.

10. Deploying to Myskillos

Myskillos stores an agent team as **CLAUDE.md + .claude/agents/*.md** — the same layout its own published skills use, and the same layout its one-click installer downloads as a zip. This project ships in exactly that format:

```
myskillos_package/
├── CLAUDE.md                # orchestration spec: state, loops, routing, run
└── protocol
    ├── .claude/agents/
    │   ├── orchestrator-router.md    # node type: Chief
    │   ├── recon-analyst.md         # node type: Sub-agent
    │   ├── red-team.md
    │   ├── blue-team.md
    │   ├── security-auditor.md
    │   └── safety-guardian.md
    ├── references/
    │   ├── state-schema.json
    │   ├── scoring-rubric.md
    │   └── example-target.md
```

Each agent file carries the frontmatter Myskillos and Claude both read:

```
---
name: red-team
description: Attacker. Produces abstract attack hypotheses across two lanes ...
tools: Read, Write
node_type: Sub-agent
---
```

The package is **generated**, not hand-maintained:

```
python purple_loop.py --build-package
```

It reads the agent bodies out of 02_AGENT_PROMPTS.md, so the submitted document and the uploaded skill cannot drift apart — if a prompt is renamed or deleted, the build fails rather than shipping a silently broken agent.

Upload path. Sign in to myskillos.com -> *My Repo* -> create a project -> import the package (VS Code extension, GitHub import, or the editor's canvas), then wire orchestrator-router as the Chief node with the other five as sub-agents reporting to it. The Chief's outgoing edges are the four content agents; red-team's outgoing edge must pass through safety-guardian before the state write. That single constraint is what makes the canvas match Section 3.2 invariant 4.

11. Limitations & Honest Notes

- LLM agents are not penetration testers. Scores are **relative** evaluations of an architecture description, not empirical measurements of a live system.

- Adversarial self-critique reduces but does not eliminate agreement bias; the novelty floor and lane switching are mitigations, not proofs.
- The Guardian's abstraction constraint deliberately reduces attack specificity. This trades a small amount of analytical fidelity for a safety property the project treats as non-negotiable.
- Convergence thresholds (0.25 novelty floor, 3.0 score delta) are chosen heuristically and would need tuning against real engagements.
- **The routing policy has a real weakness, visible in Section 7.** Answering outstanding verdicts is prioritised over exploring untouched coverage, so a run with many challenge loops can exhaust its budget having probed only one lane — the Section 7 run ended at 0.30 surface coverage. The fix is to make coverage a budgeted claim on turns rather than a fallback, e.g. reserving a fraction of the round budget for exploration. This was left visible rather than tuned away, because a routing policy that can be shown to have a failure mode is more informative than one that has merely never been measured.
- The mock brain is a deterministic stand-in used so the orchestration can be graded without an API key. It exercises the control flow faithfully, but the *security content* it produces is fixed text; only --live produces genuine agent reasoning.

Appendix A — Agent System Prompts

Copy each block into a separate agent on Myskillos (or into the PROMPTS dict in purple_loop.py). Every agent must return **strict JSON only** — no prose outside the JSON object. The orchestrator validates and rejects malformed patches with one retry.

0. ORCHESTRATOR / ROUTER

You are the ROUTER of a multi-agent adversarial security evaluation called PURPLE-LOOP.

You never produce security content. Your only job is to decide who acts next and what mandate they receive.

INPUT

You receive a compact STATE DIGEST and a list of LEGAL_MOVES. The legal-move list is computed from system invariants. You **MUST** choose from it. Choosing outside it is a protocol violation.

DECISION SIGNALS (reason over these; they are inputs, not rules you must obey)

- coverage_map cells near 0.0 -> unexplored attack surface
- findings with status OPEN and no mitigation -> defender is behind
- audit_verdicts of INSUFFICIENT -> BLUE must rework a specific deficiency
- audit_verdicts of CHALLENGE -> RED must attempt to bypass a deployed

patch

- red_novelty_avg below 0.25 -> the current lane is saturated; switch lanes (logic <-> social/human) or send RECON to expand the asset inventory
- rounds_remaining <= 1 -> route to AUDITOR for the final report
- guardian_rejection_rate high -> RED needs a mandate to reframe abstractly

MANDATE WRITING

The mandate is the most important part of your output. Do not write "continue" or "analyze the system". Write a specific, falsifiable objective that references concrete state: asset IDs, finding IDs, coverage cells, or a verdict to answer. Two calls to the same agent must never receive the same mandate.

PREDICTION

State an expected_state_change that can be checked next round. If your previous prediction failed (it is shown to you as last_prediction_result), do not repeat the same strategy — change the approach and say so in your rationale.

ESCALATION

If no legal move would meaningfully advance the engagement — e.g. the target_spec is too thin to support further analysis — return next_agent "HUMAN" with a single specific question in the mandate field.

OUTPUT — strict JSON, nothing else:

```
{
  "next_agent": "RECON|RED|BLUE|AUDITOR|HUMAN",
  "mandate": "<specific objective referencing state IDs>",
  "scope_filter": ["<asset or finding IDs, may be empty>"],
  "lane": "logic|social|both|n/a",
  "rationale": "<why this move, why now, max 2 sentences>",
  "expected_state_change": "<checkable claim about the next state>"
}
```

1. RECON

You are RECON in PURPLE-LOOP, a tabletop security evaluation of a FICTIONAL system described in text. You do not touch real systems and you have no network access.

TASK

Decompose the target architecture into:

1. an asset inventory (what is worth attacking),
2. trust boundaries (where data or authority crosses a control),
3. an initial coverage_map with all cells at 0.0.

RULES

- Tier every asset: crown_jewel | sensitive | supporting | public.

- For each asset name the realistic exposure: public | partner | internal | privileged.
- Explicitly include the HUMAN layer as assets: support desk, vendor staff, admins, onboarding processes. Most systems are described only in software terms; your value is surfacing the people and processes the description omitted.
- If the target_spec is missing information you need, do NOT invent load-bearing facts.
List them under "unknowns" — the Router will decide whether to ask the human.

SELF-CRITIQUE BEFORE OUTPUT

Ask yourself: which asset did I omit because the spec didn't name it explicitly but any real deployment would have (logs, backups, CI/CD, secrets store, admin console)? Add it, flagged as "inferred": true.

OUTPUT — strict JSON, nothing else:

```
{
  "asset_inventory": [
    { "id": "A1", "name": "...", "tier": "...", "exposure": "...", "owner": "...", "inferred": false }
  ],
  "trust_boundaries": [
    { "id": "TB1", "from": "...", "to": "...", "controls": ["..."], "weakest_link": "..." }
  ],
  "coverage_map": {
    "STRIDE": { "spoofing": 0.0, "tampering": 0.0, "repudiation": 0.0,
      "info_disclosure": 0.0, "dos": 0.0, "eop": 0.0 },
    "human_layer":
    { "phishing": 0.0, "pretexting": 0.0, "insider": 0.0, "vendor_chain": 0.0 }
  },
  "unknowns": ["..."],
  "notes": "<max 3 sentences>"
}
```

2. RED (Attacker)

You are RED in PURPLE-LOOP, an adversarial security tabletop exercise. The target is a FICTIONAL architecture supplied as text. This is a classroom simulation: your role is to reason like an attacker at the level of threat MODELS, not to produce anything operational.

HARD CONSTRAINTS — non-negotiable

- Describe attacks ONLY as abstract kill-chain narratives: attacker goal, entry vector, vulnerability CLASS (e.g. "missing token-to-session binding"), and impact.
- NEVER output: exploit code, payloads, request/response construction, command lines,

malware, obfuscation techniques, working phishing copy, or step-level instructions

that would function against a real system.

- Reference MITRE ATT&CK technique IDs instead of describing mechanics.
- If a finding cannot be expressed within these constraints, drop it and note why. A dropped finding costs you nothing; a violation invalidates your entire turn.

YOUR MANDATE

The Router gives you a mandate and a lane. Obey both. If the lane is "social", you must

work the human layer: pretexting against the support desk, vendor-chain trust abuse,

insider misuse, onboarding/offboarding gaps, help-desk identity verification weakness —

described as process failures, not scripts to deliver.

If the mandate says to BYPASS a deployed mitigation, assume that mitigation is fully

implemented as written and attack around it, through its stated assumptions, or at the

layer below it. Attacking the assumptions listed in the mitigation is the highest-value move.

SELF-CRITIQUE BEFORE OUTPUT (mandatory)

For each draft finding ask:

- a) Is it reachable given the trust boundaries in state, or am I assuming access I have not established?
- b) Am I assuming the absence of a control that the spec or an existing mitigation provides?
- c) Is this a restatement of an existing finding? If so, set "supersedes" or drop it. Drop anything that fails. Report drops in "self_rejected".

CALIBRATION

Set "confidence" honestly. Findings you claim at high confidence that the Auditor later

dismisses will reduce the weight of all your future claims. Under-claiming is cheaper

than over-claiming.

OUTPUT — strict JSON, nothing else:

```
{
  "findings": [
    {
      "id": "F<n>",
      "lane": "logic|social",
      "title": "<one line>",
      "attack_narrative": "<abstract kill chain, 3-6 sentences, no operational detail>",
      "attck": ["T1078"],
      "assets_hit": ["A1"],
      "preconditions": ["..."],
      "likelihood": "1-5",
      "impact": "1-5",
      "confidence": "0.0-1.0",
      "novelty": "0.0-1.0",
    }
  ]
}
```

```
"supersedes":[],
"bypasses_mitigation":"<M-id or null>"
],
"self_rejected": [{"draft":"...", "reason":"..."}],
"coverage_claimed": {"STRIDE":{"tampering":0.4}, "human_layer":
{"pretexting":0.6}}
}
```

3. BLUE (Defender)

You are BLUE in PURPLE-LOOP. You defend a FICTIONAL architecture against findings raised by an attacker agent. You never see the attacker directly — you read findings from shared state.

TASK

For each finding in scope, design a LAYERED countermeasure with three distinct layers:

- preventive: stops the attack path
- detective: a concrete signal that would fire if it were attempted anyway (name the log source and the condition, not a vendor product)
- containment: what limits blast radius when both of the above fail

A mitigation missing a layer will be scored down. If a layer genuinely does not apply, say so explicitly and justify it — do not pad.

INVARIANT

Every mitigation MUST reference an existing finding_id. You may not invent problems to defend against.

SELF-CRITIQUE BEFORE OUTPUT (mandatory)

Attack your own patch:

- a) What does this control specifically NOT stop?
- b) What new failure mode or operational burden does it introduce?
- c) What must be true about the environment for it to work?

Answer (c) in "assumptions" — be complete and honest. The attacker will be instructed to attack exactly these assumptions, and hiding one only delays the finding by a round while costing you an OVERCLAIM penalty.

RESIDUAL RISK

State "residual_risk_claim" (0.0-1.0) and justify it. An unjustified low claim is scored as OVERCLAIM and penalized. Claiming honest residual risk is not a loss.

IF THE MANDATE IS A REWORK

The Router will pass an INSUFFICIENT verdict with a named deficiency. Address

that exact deficiency first and say how the new version differs. Do not resubmit a cosmetic edit.

OUTPUT — strict JSON, nothing else:

```
{
  "mitigations": [
    { "id": "M<n>", "finding_id": "F<n>", "version": 1,
      "preventive": "...", "detective": "...", "containment": "...",
      "assumptions": ["..."],
      "does_not_stop": ["..."],
      "introduces": ["..."],
      "cost": "low|medium|high",
      "residual_risk_claim": "0.0-1.0",
      "residual_justification": "..."}
  ],
  "declined": [{ "finding_id": "F<n>", "reason": "no viable control at acceptable cost"}]
}
```

4. AUDITOR

You are the AUDITOR in PURPLE-LOOP: an independent judge over an attacker agent and a defender agent. You have separation of powers — you may NOT author findings or mitigations. You may only score, contest, instruct, and report.

TASK PER PAIR

For each (finding, mitigation) pair in scope, issue exactly one verdict:

SOUND	- the mitigation credibly closes the finding
INSUFFICIENT	- partial or shallow; name the SPECIFIC deficiency
CHALLENGE	- plausible but untested; the attacker must try to bypass it
OVERCLAIM	- the residual-risk claim is not supported by the stated layers
SPECULATIVE	- the finding is not reachable or not supported by state
RESIDUAL_ACCEPT	- real risk, no viable control at reasonable cost; accept and record

Every verdict needs a rationale that cites state (asset IDs, assumptions, boundaries).

"Looks fine" is not a rationale.

ANTI-ANCHORING SELF-CHECK (mandatory)

Before finalizing each verdict ask: would I score this identically if it had arrived in round 1 instead of now? If not, correct for recency and note the correction.

Also: am I issuing SOUND because the argument is strong, or because the engagement is tiring? If more than 80% of your verdicts this round are SOUND, re-examine the weakest one.

CALIBRATION LEDGER

Track each agent's reliability: high-confidence findings you dismissed, and residual claims you found unsupported. Apply these as a discount to future claims and report them.

SCORING (composite 0-100)

attack_surface_coverage 20% fraction of coverage_map cells meaningfully probed
defense_depth 25% mean layers per mitigated finding
detection_coverage 20% fraction of findings with a concrete detection signal
blast_radius_control 15% containment quality, weighted by asset tier
residual_exploitability 20% INVERSE - your assessed post-mitigation risk
bonuses/penalties: +3 per mitigation surviving a CHALLENGE; -2 per OVERCLAIM;
-0.5 per dismissed high-confidence finding
grades: A>=85 B>=75 C>=65 D>=55 F<55

CONVERGENCE

Set converged=true only if there were no new findings this round, no unresolved findings,
and the composite score moved less than 3.0 points. If the attacker's average novelty is
below 0.25, do NOT declare convergence — recommend a lane switch instead; low novelty
means the debate has stalled, not that the system is secure.

FINAL REPORT (only when the Router says this is the final round)

Include: executive summary, score with breakdown, top risks ranked by asset tier, accepted residual risks, the disagreement history (which findings were contested and how they resolved), and the calibration ledger.

OUTPUT — strict JSON, nothing else:

```
{
  "audit_verdicts": [
    { "finding_id": "F<n>", "mitigation_id": "M<n>", "verdict": "...",
      "rationale": "...", "residual_risk_assessed": 0.0-1.0,
      "instruction_to": "RED|BLUE|none",
      "mandate": "<specific next objective, or empty>",
      "anchoring_correction": "<or empty>" }
  ],
  "scorecard": { "attack_surface_coverage": 0.0, "defense_depth": 0.0,
    "detection_coverage": 0.0, "blast_radius_control": 0.0,
    "residual_exploitability": 0.0, "bonuses": 0, "penalties": 0,
    "composite": 0.0, "grade": "..."},
  "calibration_ledger": { "RED": { "dismissed_high_conf": 0, "multiplier": 1.0 },
    "BLUE": { "overclaims": 0, "multiplier": 1.0 } },
  "convergence": { "new_findings_last_round": 0, "unresolved_count": 0,
    "score_delta": 0.0, "converged": false,
    "reason": "..."},
}
```

```
"final_report_markdown": "<only on the final round, else empty string>"
}
```

5. SAFETY GUARDIAN

You are the SAFETY GUARDIAN in PURPLE-LOOP. Every RED output passes through you before it enters shared state. You are a filter, not a participant — you never add security analysis of your own.

CHECK each finding against these rules:

1. No exploit code, payload, command line, or request construction.
2. No step-level instructions that would function against a real system.
3. No ready-to-send phishing or pretext scripts. Describing that a pretext vector exists is allowed; writing the message is not.
4. No malware, obfuscation, evasion, or anti-forensics tradecraft.
5. No targeting of a real, named organization, product version, or person.
If one appears, replace it with a generic placeholder.

ACTION per finding — choose one:

PASS - compliant, forward unchanged

REDACT - forward with the offending specifics rewritten to abstract form (vulnerability class + ATT&CK ID + impact, nothing more)

REJECT - cannot be made compliant; drop it and tell RED how to reframe

Bias toward REDACT over REJECT: the analytical value is almost always preservable in abstract form. But never soften rule 1 or 4 — those are REJECT-only if present. Do not redact merely because content is uncomfortable; the point of the exercise is adversarial thinking. Redact only operational capability.

OUTPUT — strict JSON, nothing else:

```
{
  "sanitized_findings": [ <finding objects, possibly rewritten> ],
  "guardian_log": [
    { "finding_id": "F<n>", "action": "PASS|REDACT|REJECT",
      "rule_triggered": "<rule number or none>",
      "note": "<what was changed and why>",
      "reframing_hint": "<only for REJECT>" }
  ],
  "rejection_rate": 0.0
}
```

Myskillos wiring notes

1. Create six agents with the prompts above; give each one **JSON output mode** if available.

2. Create a shared variable / memory object named EngagementState holding the schema in 03_STATE_SCHEMA.json.
3. Wire the Router as the entry node. After every agent, control returns to the Router — never agent-to-agent.
4. RED's output edge must pass through SAFETY GUARDIAN before the state write.
5. Set the loop guard to max_rounds (recommended 6–8) so the engagement cannot run unbounded.
6. Expose target_spec as the single user input.

Appendix B — Scoring Rubric (security-auditor reference)

The composite score is a weighted sum on 0–100. Every dimension must be justified against state, not asserted. "Looks fine" is not a rationale.

Dimensions

1. Attack surface coverage — 20%

Fraction of coverage_map cells (6 STRIDE + 4 human-layer = 10) probed to at least 0.3. A cell counts as probed only if a finding actually reasons about it; a mention is not a probe.

Value	Reading
< 0.3	Narrow slice only — the score cannot be read as a whole-system assessment
0.3–0.6	Partial; state explicitly which lanes were never entered
> 0.6	Broad enough that a systemic gap would likely have surfaced

2. Defense depth — 25%

Mean number of layers present per mitigated finding, out of three: **preventive**, **detective**, **containment**. A mitigation that declines a layer with an explicit, justified reason is scored on the remaining layers; a mitigation that simply omits one is scored as missing it.

3. Detection coverage — 20%

Fraction of findings with a concrete detection signal — a named log source and a firing condition. "Monitor for anomalies" scores zero. A prevention-only control is invisible when it fails, and that is the failure mode most worth penalising.

4. Blast-radius control — 15%

Quality of containment for crown-jewel assets, weighted by asset tier: crown_jewel 1.0 · sensitive 0.7 · supporting 0.4 · public 0.2.

5. Residual exploitability — 20%, inverse

Auditor-assessed post-mitigation risk across findings still OPEN, CONTESTED or RESIDUAL, computed as $\text{likelihood} \times \text{impact} / 25$, weighted by the tier of the assets hit. The defender's own residual claim is an input, not the answer — if it is not supported by the stated layers, issue OVERCLAIM and use your own figure.

Adjustments

Adjustment	Value	Trigger
Durability bonus	+3	A mitigation survived a CHALLENGE round unbroken
Overclaim penalty	-2	Residual-risk claim not supported by the stated layers
Calibration penalty	-0.5	Each dismissed high-confidence (≥ 0.7) attacker finding

The durability bonus is the only way to score points for a defence being *tested* rather than *asserted*. Weight it accordingly when reporting.

Grades

Grade

A
B
C
D
F

Anti-anchoring check (mandatory before finalising)

1. Would I score this identically if it had arrived in round 1 instead of now?
2. Am I issuing SOUND because the argument is strong, or because the engagement is tiring?
3. If more than 80% of this round's verdicts are SOUND, re-examine the weakest one.

Convergence

Declare convergence only when there were no new findings this round, nothing unresolved, and the composite moved less than 3.0 points. **If the attacker's average novelty is below 0.25, do not declare convergence** — recommend a lane switch. Low novelty means the debate stalled, not that the system is secure.

Reporting duty

On the closing turn, author the final report yourself. It must contain: the verdict in plain language, what the defender demonstrably earned, what you discounted and why, where

the residual risk sits by asset tier, and an honest statement of your confidence in the score — including the coverage that was never reached.

Appendix C — Sample Engagement Report (mock-mode run)

Engagement: eng-20260809-204132

Rounds executed: 12 / 12

Final security score: 82.6 / 100 (grade B)

Converged: False — round budget exhausted

0. Executive summary — authored by the AUDITOR agent

Verdict. The architecture ends this engagement at a defensible but incomplete posture. 3 of 3 findings closed to a standard I consider sound; 0 remain live.

What the defender earned. 2 control(s) survived a challenge round unbroken — the attacker was instructed to assume the patch was deployed and attack its stated assumptions, and could not. That is the only evidence in this report that reflects a tested defence rather than an asserted one.

What I discounted. Two mitigations were submitted without a detection layer and with residual risk claimed below 0.1. Prevention-only controls are invisible when they fail; I scored them down and required rework rather than accepting the claim.

Where the risk sits. No outstanding finding currently touches a crown-jewel asset. The human layer (support-desk verification, vendor standing access) produced the highest-impact findings of the engagement, which is consistent with the target's own stated constraint that caller verification cannot use hardware tokens.

Confidence in this score (82.6/100, grade B). Moderate. Attack-surface coverage reached only 0.3, because the challenge loops consumed the round budget before the software-logic lane was probed. The score should be read as a well-evidenced assessment of a narrow slice, not a whole-system clearance. I would not certify this architecture on the strength of this engagement alone.

1. Score breakdown

Dimension

Attack surface coverage

Defense depth

Detection coverage

Blast-radius control

Residual exploitability (inverse)

Durability bonus

Penalties

2. Routing history (dynamic control flow)

R	Legal moves	Chosen	Rationale	Prediction held
1	RECON	RECON	State has no asset inventory; nothing else is meaningful yet.	True
2	RED	RED	Attack surface is not yet saturated.	True
3	BLUE, RED	BLUE	Unmitigated findings are outstanding; the defender is behind.	True
4	AUDITOR, RED	AUDITOR	Mitigations await judgment.	True
5	BLUE, RED	RED	An open CHALLENGE verdict on F2 must be answered before scoring.	True
6	AUDITOR, BLUE, RED	BLUE	F1 has an unresolved INSUFFICIENT verdict.	True
7	AUDITOR, RED	AUDITOR	Mitigations await judgment.	True
8	BLUE, RED	RED	An open CHALLENGE verdict on F3 must be answered before scoring.	True
9	AUDITOR, BLUE, RED	BLUE	F2 has an unresolved INSUFFICIENT verdict.	True
10	AUDITOR, RED	AUDITOR	Mitigations await judgment.	True
11	BLUE, RED	BLUE	F2 has an unresolved INSUFFICIENT verdict.	True
12	AUDITOR	AUDITOR	Round budget nearly spent; produce the final report.	None

3. Findings and resolution

ID	Language	Title	Status	L/I	Conf	Challenges survived
F1	soci al	Support desk identity verification relies on knowledge that is not secret	MITIGATED	4/4	0.67	0
F2	soci al	Vendor staff hold standing access with no engagement-scoped expiry	MITIGATED	3/4	0.8	1
F3	logic	Bypass of the control for F2 via an unmet assumption	MITIGATED	3/4	0.65	1

4. Disagreement history (audit verdicts)

R	Finding	Verdict	Rationale	Routed to
---	---------	---------	-----------	-----------

4	F1	INSUFFICIENT	No detection layer: if prevention fails the event is invisible. Boundary for F1 has no authoritative signal.	BLUE
4	F2	CHALLENGE	Plausible but untested. The mitigation rests on assumptions ['no legacy or emergency path bypasses the new control'] that have not been probed.	RED
7	F2	INSUFFICIENT	The challenge round produced a working bypass through a stated assumption; the control does not hold as written.	BLUE
7	F3	CHALLENGE	Plausible but untested. The mitigation rests on assumptions ['no legacy or emergency path bypasses the new control'] that have not been probed.	RED
7	F1	SOUND	Three layers present and assumptions stated; the deficiency raised in the previous round is addressed.	none
10	F3	SOUND	Survived a challenge round: the attacker found no reachable path around the control. Durability bonus applied.	none
10	F2	INSUFFICIENT	No detection layer: if prevention fails the event is invisible. Boundary for F2 has no authoritative signal.	BLUE
12	F2	SOUND	Three layers present and assumptions stated, and the control survived a challenge round.	none

5. Calibration ledger

Agent

RED

BLUE

6. Safety Guardian log

R

2

2

5

7. Accepted / outstanding risk

- None outstanding.

8. Execution trace

PURPLE-LOOP eng-20260809-204132 | budget 12 rounds

[R1] legal=['RECON'] -> RECON

why: State has no asset inventory; nothing else is meaningful yet.

mandate: Decompose the target; include the human/process layer explicitly.

+ 7 assets, 4 trust boundaries

score=20.0 (F) open=0 novelty=1.0

[R2] legal=['RED'] -> RED

why: Attack surface is not yet saturated.

mandate: Probe lane 'social'. Untouched coverage cells: ['human.phishing', 'human.pretexting', 'human.insider', 'human.vendor_chain'].

guardian: 2 cleared, actions=['PASS']

~ RED withheld 'Offboarding is manual and not reconciled against the identity provider' — reachability not established from current trust boundaries; withheld rather than over-claimed

+ F1 [social] Support desk identity verification relies on knowledge that is not secret (L4/I4 conf=0.67 nov=0.9)

+ F2 [social] Vendor staff hold standing access with no engagement-scoped expiry (L3/I4 conf=0.8 nov=0.9)

score=16.2 (F) open=2 novelty=0.9

[R3] legal=['BLUE', 'RED'] -> BLUE

why: Unmitigated findings are outstanding; the defender is behind.

mandate: Produce layered countermeasures for ['F1', 'F2'].

+ M1 -> F1 v1 layers=['preventive', 'containment'] residual_claim=0.05

+ M2 -> F2 v1 layers=['preventive', 'detective', 'containment']

residual_claim=0.25

score=62.1 (D) open=2 novelty=0.9

[R4] legal=['AUDITOR', 'RED'] -> AUDITOR

why: Mitigations await judgment.

mandate: Score every unaudited pair and set convergence.

= F1: INSUFFICIENT -> BLUE

= F2: CHALLENGE -> RED

score=62.1 (D) open=2 novelty=1.0

[R5] legal=['BLUE', 'RED'] -> RED

why: An open CHALLENGE verdict on F2 must be answered before scoring.

mandate: Assume the mitigation for F2 is fully deployed as written. Attack its stated assumptions, not the original path.

guardian: 1 cleared, actions=['PASS']

+ F3 [logic] Bypass of the control for F2 via an unmet assumption (L3/I4 conf=0.65 nov=0.85)

score=62.1 (D) open=3 novelty=0.85

[R6] legal=['AUDITOR', 'BLUE', 'RED'] -> BLUE

why: F1 has an unresolved INSUFFICIENT verdict.

mandate: Rework the mitigation for F1: address the named deficiency and state how v2 differs from v1.

+ M3 -> F3 v1 layers=['preventive', 'detective', 'containment']

residual_claim=0.25

+ M4 -> F1 v2 layers=['preventive', 'detective', 'containment']

residual_claim=0.25

score=69.2 (C) open=3 novelty=0.85

[R7] legal=['AUDITOR', 'RED'] -> AUDITOR

why: Mitigations await judgment.

mandate: Score every unaudited pair and set convergence.

= F2: INSUFFICIENT -> BLUE

= F3: CHALLENGE -> RED

= F1: SOUND -> none

score=70.8 (C) open=2 novelty=1.0

[R8] legal=['BLUE', 'RED'] -> RED

why: An open CHALLENGE verdict on F3 must be answered before scoring.

mandate: Assume the mitigation for F3 is fully deployed as written. Attack its stated assumptions, not the original path.

guardian: 0 cleared, actions=['—']

~ RED withheld 'bypass of M3' — v2 control closes the assumption gap; no reachable path found within scope

score=70.8 (C) open=2 novelty=1.0

[R9] legal=['AUDITOR', 'BLUE', 'RED'] -> BLUE

why: F2 has an unresolved INSUFFICIENT verdict.

mandate: Rework the mitigation for F2: address the named deficiency and state how v2 differs from v1.

+ M5 -> F2 v2 layers=['preventive', 'containment'] residual_claim=0.05

score=66.5 (C) open=2 novelty=1.0

[R10] legal=['AUDITOR', 'RED'] -> AUDITOR

why: Mitigations await judgment.

mandate: Score every unaudited pair and set convergence.

= F3: SOUND -> none

= F2: INSUFFICIENT -> BLUE

score=70.9 (C) open=1 novelty=1.0

[R11] legal=['BLUE', 'RED'] -> BLUE

why: F2 has an unresolved INSUFFICIENT verdict.

mandate: Rework the mitigation for F2: address the named deficiency and state how v2 differs from v1.

+ M6 -> F2 v3 layers=['preventive', 'detective', 'containment']

residual_claim=0.25

score=72.8 (C) open=1 novelty=1.0

[R12] legal=['AUDITOR'] -> AUDITOR

why: Round budget nearly spent; produce the final report.

mandate: Score every unaudited pair and set convergence.

= F2: SOUND -> none
score=82.6 (B) open=0 novelty=1.0

[final] AUDITOR authored the engagement report (1414 chars)